

PLANO COMPLETO DE DESENVOLVIMENTO: PWA GUIA DE DOENÇAS EM GALINHAS

Data: 10 de Abril de 2026

Versão: 1.0 - FINAL

Objetivo: Criar um Progressive Web App (PWA) profissional, offline-first, com busca inteligente, compatível com iOS/Android/Windows



ÍNDICE

1. [Visão Geral do Projeto](#)
 2. [Requisitos Técnicos Completos](#)
 3. [Arquitetura da Aplicação](#)
 4. [Estrutura de Dados](#)
 5. [Especificações Detalhadas de Funcionalidades](#)
 6. [Stack Tecnológico](#)
 7. [Estrutura de Pastas e Arquivos](#)
 8. [Componentes React Necessários](#)
 9. [Configuração PWA \(Service Worker, Manifest\)](#)
 10. [Guia de Implementação Passo a Passo](#)
 11. [Testes e Validação](#)
 12. [Instruções de Compartilhamento para Usuários](#)
 13. [Estratégia de Atualização](#)
-

VISÃO GERAL DO PROJETO

Objetivo Principal

Transformar informações sobre doenças em galinhas (originalmente em PDF) em um **Progressive Web App interativo**, que funcione **100% offline**, seja facilmente compartilhável via link/WhatsApp/email, e se atualize automaticamente quando novas informações forem disponibilizadas.

Público-Alvo

Produtores rurais com experiência limitada em tecnologia, usando dispositivos iOS, Android ou Windows.

Requisitos Críticos

-  Funciona **100% offline** após primeira instalação
-  Compatível com **iOS (Safari), Android (Chrome), Windows (Edge/Chrome)**
-  Compartilhável via **link único** (sem arquivo para download)
-  Instalável na tela inicial como app nativo
-  Busca inteligente com fuzzy matching
-  Atualização automática de conteúdo
-  Interface intuitiva para usuários rurais
-  Sem dependência de servidor após instalação

REQUISITOS TÉCNICOS COMPLETOS

Para Desenvolvimento

- Node.js 18+ com npm/pnpm
- React 19 com TypeScript
- Tailwind CSS 4

- shadcn/ui (componentes pré-construídos)
- Vite (build tool)
- Service Worker API
- IndexedDB
- Fuse.js (busca fuzzy)

Para Deployment

- Servidor web com suporte a HTTPS (obrigatório para PWA)
- CDN para servir assets estáticos
- Domínio customizado (recomendado: galinhas.manus.space ou similar)

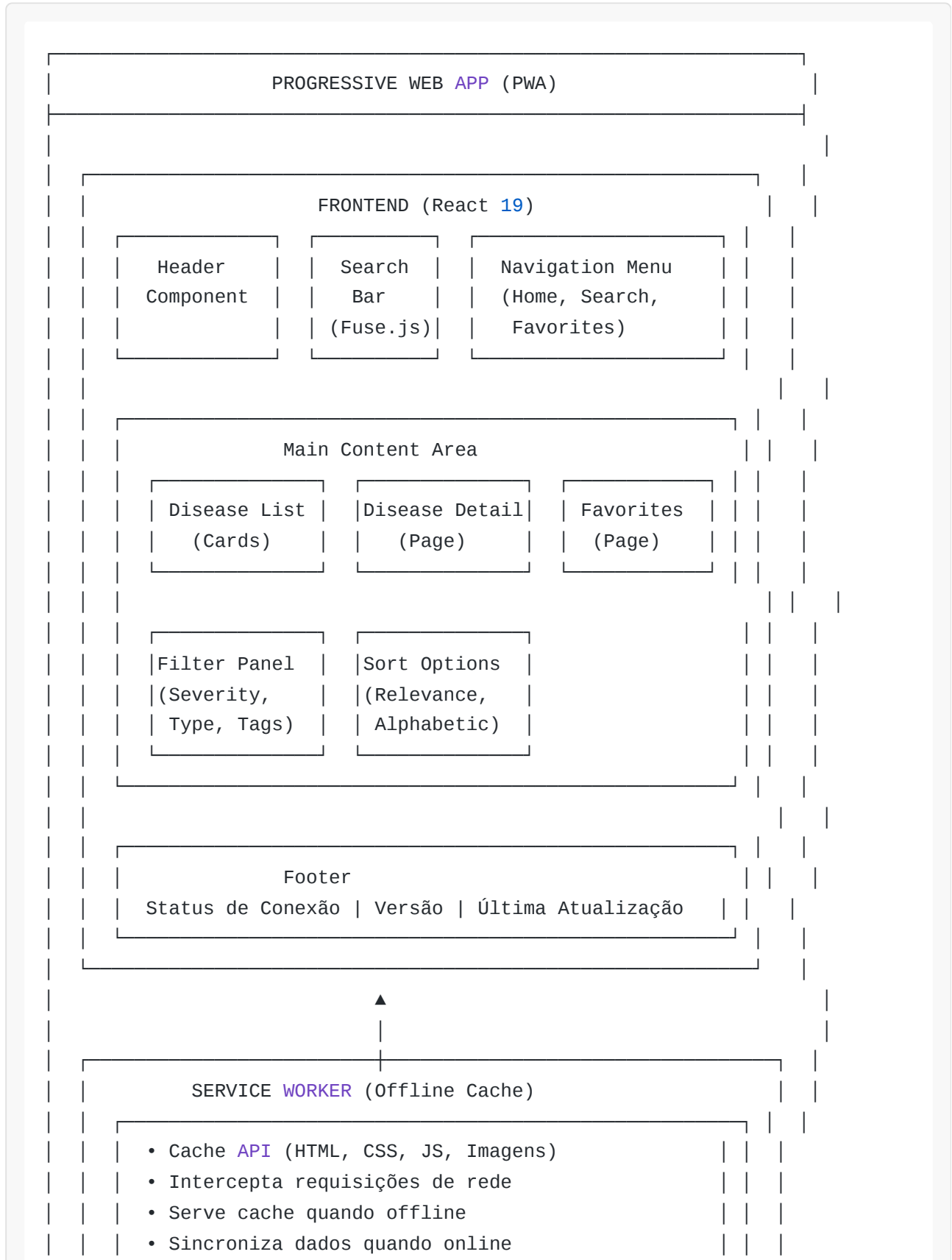
Requisitos de Navegadores

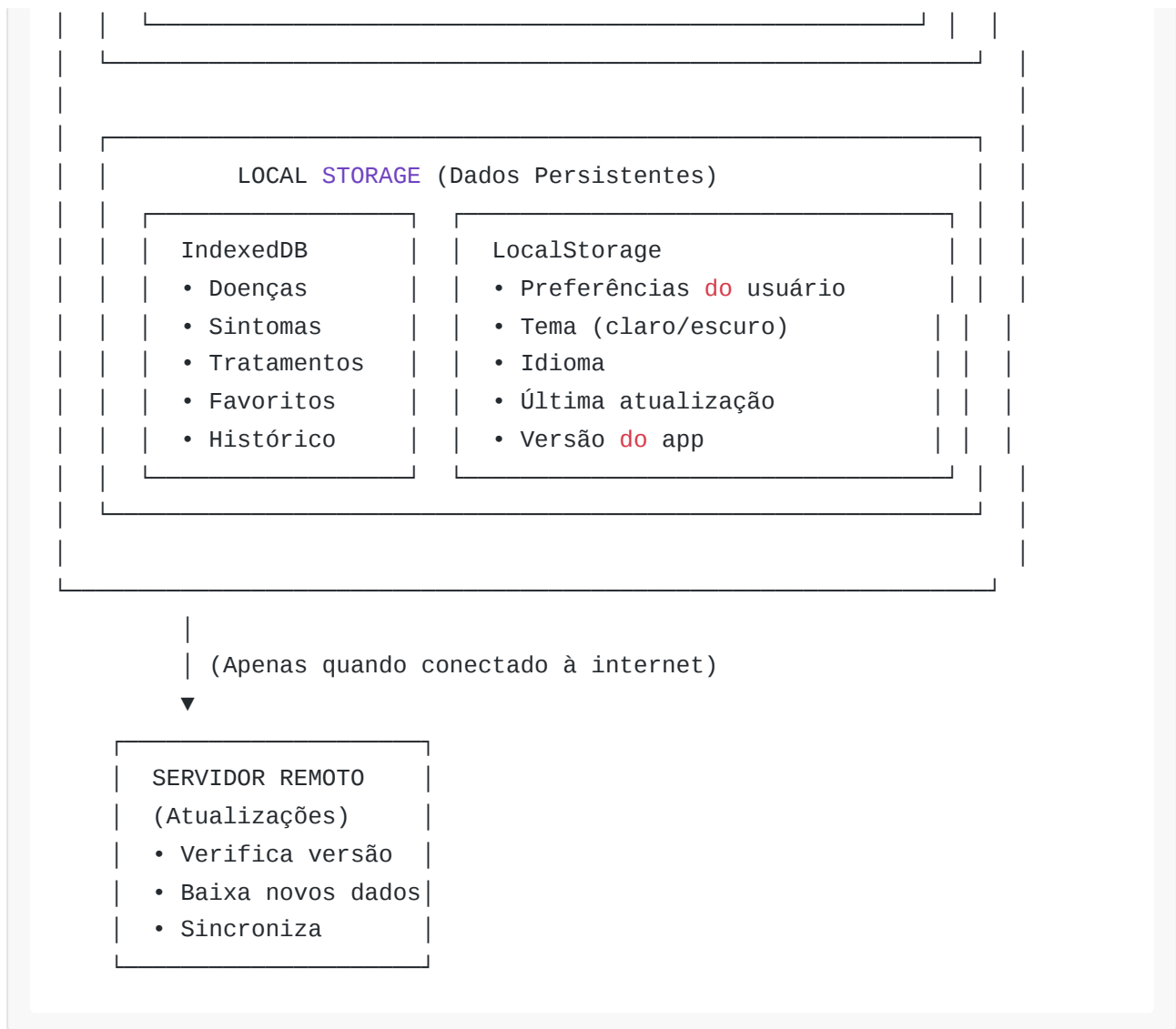
Navegador	Versão Mínima	Suporte PWA
Safari (iOS)	11.3+	✓ Sim (com limitações)
Chrome (Android)	40+	✓ Sim (completo)
Edge (Windows)	79+	✓ Sim (completo)
Firefox	55+	✓ Sim (completo)



ARQUITETURA DA APLICAÇÃO

Diagrama de Arquitetura





Fluxo de Dados

1. **Primeira Visita:** Usuário acessa o link → Service Worker faz cache de todos os recursos → IndexedDB carrega dados das doenças
 2. **Uso Offline:** Usuário usa a app → Service Worker serve cache → Busca funciona com dados locais
 3. **Online:** Service Worker verifica atualizações → Se houver, baixa novos dados → Notifica usuário
 4. **Atualização:** Usuário clica “Atualizar” → Novos dados são carregados → App recarrega
-



ESTRUTURA DE DADOS

Formato JSON de Uma Doença

```
{
  "id": "doenca-001",
  "nome": "Coccidiose",
  "nomeAlternativo": ["Coccidióse", "Eimeriose"],
  "descricaoBreve": "Infecção parasitária que afeta o intestino das galinhas, causada pelo protozoário Eimeria",
  "descricaoCompleta": "A coccidiose é uma doença parasitária que afeta principalmente o intestino delgado das galinhas. É causada por protozoários do gênero Eimeria e é uma das doenças mais comuns em criações avícolas...",
  "severidade": "alta",
  "tipo": "parasitaria",
  "faixaEtaria": ["pintinhos", "galinhas-jovens"],
  "sintomas": [
    {
      "nome": "Diarreia sanguinolenta",
      "descricao": "Fezes com sangue, geralmente nos primeiros 5-7 dias de infecção",
      "gravidade": "alta"
    },
    {
      "nome": "Apatia",
      "descricao": "Falta de disposição, galinha fica quieta e isolada",
      "gravidade": "media"
    },
    {
      "nome": "Perda de apetite",
      "descricao": "Galinha não come ou come muito pouco",
      "gravidade": "media"
    },
    {
      "nome": "Penas eriçadas",
      "descricao": "Penas arrepiadas, dando aparência despenteada",
      "gravidade": "baixa"
    },
    {
      "nome": "Retardo no crescimento",
      "descricao": "Filhotes crescem mais lentamente",
      "gravidade": "media"
    }
  ]
},
```

```
"causas": [  
  {  
    "causa": "Parasita Eimeria",  
    "descricao": "Protozoário que invade as células intestinais"  
  },  
  {  
    "causa": "Ambiente úmido",  
    "descricao": "Oocistos (ovos do parasita) sobrevivem melhor em  
ambientes com alta umidade"  
  },  
  {  
    "causa": "Superlotação",  
    "descricao": "Muitas galinhas em pouco espaço aumenta a transmissão"  
  },  
  {  
    "causa": "Falta de higiene",  
    "descricao": "Galinheiro sujo facilita a propagação"  
  }  
,  
  ],  
  "tratamento": {  
    "medicamentos": [  
      {  
        "nome": "Amprólio",  
        "dosagem": "125 mg/kg de ração",  
        "duracao": "7 dias",  
        "notas": "Medicamento mais comum e eficaz"  
      },  
      {  
        "nome": "Toltrazurila",  
        "dosagem": "25 mg/kg",  
        "duracao": "2-3 dias",  
        "notas": "Ação mais rápida"  
      }  
    ],  
    ],  
    "cuidados": [  
      {  
        "cuidado": "Manter o ambiente seco",  
        "descricao": "Trocar cama frequentemente, melhorar drenagem"  
      },  
      {  
        "cuidado": "Melhorar ventilação",  
        "descricao": "Aumentar circulação de ar para reduzir umidade"  
      },  
      {  
        "cuidado": "Isolar aves afetadas",  
        "descricao": "Separar galinhas doentes para evitar contaminação"
```

```
,
{
  "cuidado": "Aumentar temperatura",
  "descricao": "Manter aquecimento adequado para pintinhos"
}
],
"prevenção": [
  {
    "metodo": "Vacinação preventiva",
    "descricao": "Vacinas vivas atenuadas podem ser usadas em pintinhos"
  },
  {
    "metodo": "Higiene rigorosa",
    "descricao": "Limpeza diária do galinheiro e comedouros"
  },
  {
    "metodo": "Controle de umidade",
    "descricao": "Manter umidade relativa entre 60-70%"
  },
  {
    "metodo": "Densidade adequada",
    "descricao": "Não superlotação – máximo 8-10 galinhas por m²"
  }
]
},
"imagens": [
  {
    "url": "https://cdn.example.com/coccidiose-1.jpg",
    "descricao": "Galinha com sintomas de coccidiose",
    "credito": "Fonte: Manual de Saúde Avícola"
  },
  {
    "url": "https://cdn.example.com/coccidiose-2.jpg",
    "descricao": "Fezes com sangue característica da coccidiose",
    "credito": "Fonte: Universidade de Agricultura"
  }
],
"referencias": [
  {
    "titulo": "Manual de Saúde Avícola",
    "autor": "Dr. João Silva",
    "ano": 2023,
    "url": "https://example.com/manual"
  }
],
"tags": ["parasitaria", "intestinal", "comum", "pintinhos"],
```



```
"dataAdicao": "2026-04-10",
"dataAtualizacao": "2026-04-10",
"versao": 1
}
```

Arquivo de Configuração de Versão

```
{
  "versao": "1.0.0",
  "dataAtualizacao": "2026-04-10T10:30:00Z",
  "totalDoencas": 45,
  "idioma": "pt-BR",
  "checksum": "abc123def456",
  "novasAtualizacoes": {
    "versao": "1.1.0",
    "dataDisponivel": "2026-04-15T10:30:00Z",
    "descricao": "Adicionadas 5 novas doenças e melhorias na busca"
  }
}
```



ESPECIFICAÇÕES DETALHADAS DE FUNCIONALIDADES

1. Busca Inteligente com Fuzzy Matching

Biblioteca: Fuse.js

Comportamento:

- Busca em tempo real enquanto o usuário digita
- Busca em múltiplos campos: nome, sintomas, tratamentos, tags
- Fuzzy matching: “coccidose” encontra “coccidiose”
- Resultado ordenado por relevância
- Máximo 50 resultados por página

Campos Pesquisáveis:

```
const searchOptions = {
  keys: [
    { name: "nome", weight: 0.4 },
    { name: "nomeAlternativo", weight: 0.3 },
    { name: "descricaoBreve", weight: 0.2 },
    { name: "sintomas.nome", weight: 0.3 },
    { name: "tratamento.medicamentos.nome", weight: 0.2 },
    { name: "tags", weight: 0.2 }
  ],
  threshold: 0.3,
  minMatchCharLength: 2
};
```

Exemplo de Uso:

- Usuário digita: “diar” → Encontra “Diarreia”, “Coccidiose” (que tem diarreia)
- Usuário digita: “antibiotico” → Encontra doenças tratáveis com antibióticos
- Usuário digita: “pintinhos” → Encontra doenças que afetam pintinhos

2. Filtros Avançados

Filtros Disponíveis:

Filtro	Opções	Tipo
Severidade	Leve, Moderada, Alta	Checkbox
Tipo	Parasitária, Viral, Bacteriana, Fúngica, Nutricional	Checkbox
Faixa Etária	Pintinhos, Galinhas Jovens, Adultas	Checkbox
Ordenação	Relevância, A-Z, Z-A, Mais Recente	Select

Comportamento:

- Filtros funcionam em combinação (AND logic)
- Atualização em tempo real
- Mostrar número de resultados
- Opção “Limpar Filtros”

3. Página de Detalhes da Doença

Conteúdo:

- Nome e nome alternativo
- Descrição completa
- Ícone de severidade (visual)
- Sintomas com descrição e gravidade
- Causas
- Tratamento (medicamentos, cuidados, prevenção)
- Imagens (se disponível)
- Referências
- Botão “Adicionar aos Favoritos”
- Botão “Compartilhar”

Interações:

- Scroll suave
- Botão “Voltar” no topo
- Expandir/Colapsar seções
- Copiar texto para clipboard

4. Sistema de Favoritos

Funcionalidade:

- Marcar/desmarcar doença como favorita
- Armazenar em LocalStorage
- Página dedicada com todos os favoritos
- Sincronizar entre abas do navegador
- Ícone de coração preenchido/vazio

Dados Armazenados:

```
{
  "favoritos": [
    {
      "id": "doenca-001",
      "dataMarcada": "2026-04-10T10:30:00Z"
    }
  ]
}
```

5. Histórico de Consultas

Funcionalidade:

- Registrar cada doença visualizada
- Mostrar últimas 20 consultas
- Ordenar por data (mais recente primeiro)
- Opção de limpar histórico
- Sincronizar entre abas

Dados Armazenados:

```
{
  "historico": [
    {
      "id": "doenca-001",
      "nome": "Coccidiose",
      "dataConsulta": "2026-04-10T10:30:00Z"
    }
  ]
}
```

6. Modo Offline 100%

Funcionalidade:

- Service Worker faz cache de todos os recursos
- Indicador visual de status de conexão (online/offline)

- Mensagem amigável quando offline
- Funcionalidade completa sem internet
- Sincronização automática quando reconecta

Recursos em Cache:

- HTML, CSS, JavaScript (app shell)
- Todas as imagens
- Dados JSON das doenças
- Ícones e fontes

7. Atualização Automática

Fluxo:

1. Service Worker verifica versão a cada 24 horas
2. Se houver versão nova, baixa em background
3. Notifica usuário: “Novas informações disponíveis!”
4. Usuário clica “Atualizar” ou “Depois”
5. Se clicar “Atualizar”, recarrega a página
6. Novos dados são carregados

Implementação:

```
// Verificar atualizações a cada 24 horas
setInterval(checkForUpdates, 24 * 60 * 60 * 1000);

async function checkForUpdates() {
  const response = await fetch('/api/version.json');
  const remoteVersion = await response.json();
  const localVersion = JSON.parse(localStorage.getItem('appVersion'));

  if (remoteVersion.versao > localVersion.versao) {
    showUpdateNotification();
  }
}
```

8. Responsividade

Breakpoints:

- Mobile: 320px - 640px
- Tablet: 641px - 1024px
- Desktop: 1025px+

Comportamento:

- Mobile: Layout em coluna única, menu hambúrguer
- Tablet: Layout adaptado, sidebar colapsável
- Desktop: Layout completo com sidebar

9. Acessibilidade

Implementações:

- Contraste WCAG AA
- Labels em todos os inputs
- Navegação por teclado
- ARIA labels apropriados
- Texto alternativo em imagens
- Suporte a screen readers

10. Temas (Claro/Escuro)

Funcionalidade:

- Detectar preferência do sistema
 - Permitir alternância manual
 - Armazenar preferência em LocalStorage
 - Aplicar tema a todos os componentes
 - Transição suave entre temas
-

Frontend

Tecnologia	Versão	Propósito
React	19+	Framework UI
TypeScript	5.3+	Tipagem estática
Tailwind CSS	4+	Styling
shadcn/ui	Latest	Componentes pré-construídos
Fuse.js	7+	Busca fuzzy
Wouter	3+	Roteamento cliente
Lucide React	Latest	Ícones
Framer Motion	12+	Animações
Sonner	2+	Notificações toast
Zod	4+	Validação de dados

Build & Dev

Tecnologia	Versão	Propósito
Vite	7+	Build tool
pnpm	10+	Package manager
TypeScript	5.3+	Compilação
Tailwind CSS	4+	CSS framework

APIs Nativas (Browser)

API	Propósito
Service Worker API	Cache offline
Cache API	Armazenar recursos
IndexedDB	Banco de dados local
LocalStorage	Preferências do usuário
Fetch API	Requisições HTTP
Notification API	Notificações do sistema

ESTRUTURA DE PASTAS E ARQUIVOS

guia-doencas-galinhas-pwa/

	└─ client/					
		└─ public/				
			└─ favicon.ico	← Ícone da app		
			└─ manifest.json	← Configuração PWA (VER SEÇÃO 5)		
			└─ robots.txt	← SEO		
			└─ service-worker.js	← Service Worker (VER SEÇÃO 5)		
			└─ app-icon-192.png	← Ícone 192x192		
			└─ app-icon-512.png	← Ícone 512x512		
			└─ splash-screen-ios.png	← Splash screen iOS		
			└─ offline.html	← Página offline (fallback)		
		└─ src/				
			└─ pages/			
				└─ Home.tsx	← Página inicial	
				└─ Search.tsx	← Página de busca	
				└─ DiseaseDetail.tsx	← Detalhes de uma doença	
				└─ Favorites.tsx	← Página de favoritos	
				└─ History.tsx	← Histórico de consultas	
				└─ Settings.tsx	← Configurações	
				└─ NotFound.tsx	← Página 404	
			└─ components/			
				└─ Header.tsx	← Cabeçalho com logo e menu	
				└─ Navigation.tsx	← Menu de navegação	
				└─ SearchBar.tsx	← Barra de busca	
				└─ DiseaseCard.tsx	← Card de doença (lista)	
				└─ DiseaseDetailView.tsx	← Visualização detalhada	
				└─ FilterPanel.tsx	← Painel de filtros	
				└─ SortOptions.tsx	← Opções de ordenação	
				└─ FavoriteButton.tsx	← Botão favoritar	
				└─ ShareButton.tsx	← Botão compartilhar	
				└─ ConnectionStatus.tsx	← Indicador de conexão	
				└─ UpdateNotification.tsx	← Notificação de atualização	
				└─ Footer.tsx	← Rodapé	
				└─ LoadingSpinner.tsx	← Spinner de carregamento	
				└─ ErrorBoundary.tsx	← Tratamento de erros	
				└─ ui/	← shadcn/ui components	
					└─ button.tsx	
					└─ card.tsx	
					└─ input.tsx	

- select.tsx - checkbox.tsx - dialog.tsx - toast.tsx [outros componentes]	
- hooks/ - useDiseases.ts - useFavorites.ts - useSearch.ts - useHistory.ts - useOnline.ts - useTheme.ts - useLocalStorage.ts	← Hook para carregar doenças ← Hook para gerenciar favoritos ← Hook para busca ← Hook para histórico ← Hook para status de conexão ← Hook para tema ← Hook para localStorage
- lib/ - db.ts - diseases.ts - search.ts - storage.ts - utils.ts - constants.ts - types.ts	← Setup IndexedDB ← Dados das doenças (JSON) ← Lógica de busca (Fuse.js) ← Funções de storage ← Funções utilitárias ← Constantes da app ← Tipos TypeScript
- contexts/ - ThemeContext.tsx - DiseasesContext.tsx - UpdateContext.tsx	← Context de tema ← Context de doenças ← Context de atualizações
- App.tsx - main.tsx - index.css	← Componente raiz ← Entry point ← Estilos globais
- index.html - tsconfig.json - vite.config.ts	← HTML principal ← Configuração TypeScript ← Configuração Vite
- public/ - diseases.json	← Dados das doenças (JSON)
- package.json - pnpm-lock.yaml - tailwind.config.ts - postcss.config.js - tsconfig.json - README.md	← Dependências ← Lock file ← Configuração Tailwind ← Configuração PostCSS ← Configuração TypeScript global ← Documentação



COMPONENTES REACT NECESSÁRIOS

1. Header.tsx

```
// Exibe logo, título, e botão de menu
// Props: onClick, isOnline
// Conteúdo: Logo, "Guia de Doenças em Galinhas", ícone de menu
```

2. Navigation.tsx

```
// Menu de navegação principal
// Links: Home, Busca, Favoritos, Histórico, Configurações
// Responsivo: Menu hambúrguer em mobile, sidebar em desktop
```

3. SearchBar.tsx

```
// Barra de busca com autocomplete
// Props: onSearch, placeholder
// Funcionalidade: Busca em tempo real, limpar, ícone de busca
```

4. DiseaseCard.tsx

```
// Card exibindo resumo de uma doença
// Props: disease, onFavorite, onClick
// Conteúdo: Nome, descrição breve, severidade, tags
// Ações: Clique para detalhes, botão favoritar
```

5. DiseaseDetailView.tsx

```
// Visualização completa de uma doença
// Props: disease, onFavorite, onShare
// Conteúdo: Tudo do JSON (sintomas, causas, tratamento, imagens)
// Ações: Favoritar, compartilhar, voltar
```

6. FilterPanel.tsx

```
// Painel de filtros
// Filtros: Severidade, Tipo, Faixa Etária
// Props: onFilterChange, filters
// Ações: Aplicar, Limpar
```

7. SortOptions.tsx

```
// Opções de ordenação
// Opções: Relevância, A-Z, Z-A, Mais Recente
// Props: onSortChange, currentSort
```

8. FavoriteButton.tsx

```
// Botão para marcar/desmarcar favorito
// Props: diseaseId, isFavorite, onToggle
// Visual: Ícone de coração preenchido/vazio
```

9. ShareButton.tsx

```
// Botão para compartilhar doença
// Props: disease
// Ações: Copiar link, WhatsApp, Email, Nativa (se disponível)
```

10. ConnectionStatus.tsx

```
// Indicador de status de conexão  
// Props: isOnline  
// Visual: Ícone + texto "Online" ou "Offline"  
// Posição: Canto inferior direito ou footer
```

11. UpdateNotification.tsx

```
// Notificação de atualização disponível  
// Props: onUpdate, onDismiss  
// Conteúdo: "Novas informações disponíveis"  
// Ações: Atualizar, Depois
```

12. Footer.tsx

```
// Rodapé da app  
// Conteúdo: Status de conexão, versão, última atualização
```

CONFIGURAÇÃO PWA (SERVICE WORKER,

MANIFEST)

manifest.json

```
{
  "name": "Guia de Doenças em Galinhas",
  "short_name": "Doenças Galinhas",
  "description": "Guia completo e offline de doenças em galinhas com busca inteligente",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait-primary",
  "theme_color": "#2d5016",
  "background_color": "#ffffff",
  "categories": ["productivity", "reference"],
  "screenshots": [
    {
      "src": "/screenshot-1.png",
      "sizes": "540x720",
      "type": "image/png",
      "form_factor": "narrow"
    },
    {
      "src": "/screenshot-2.png",
      "sizes": "1280x720",
      "type": "image/png",
      "form_factor": "wide"
    }
  ],
  "icons": [
    {
      "src": "/app-icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/app-icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    }
  ]
}
```

```
        "src": "/app-icon-maskable.png",
        "sizes": "192x192",
        "type": "image/png",
        "purpose": "maskable"
    }
],
"shortcuts": [
    {
        "name": "Buscar Doença",
        "short_name": "Buscar",
        "description": "Abre a página de busca",
        "url": "/search",
        "icons": [
            {
                "src": "/icon-search.png",
                "sizes": "192x192"
            }
        ]
    },
    {
        "name": "Meus Favoritos",
        "short_name": "Favoritos",
        "description": "Abre a página de favoritos",
        "url": "/favorites",
        "icons": [
            {
                "src": "/icon-favorites.png",
                "sizes": "192x192"
            }
        ]
    }
],
"share_target": {
    "action": "/share",
    "method": "POST",
    "enctype": "application/x-www-form-urlencoded",
    "params": {
        "title": "title",
        "text": "text",
        "url": "url"
    }
}
}
```


service-worker.js

```
// IMPORTANTE: Este arquivo deve estar em client/public/

const CACHE_NAME = 'doencas-galinhas-v1';
const RUNTIME_CACHE = 'doencas-galinhas-runtime-v1';
const ASSETS_TO_CACHE = [
  '/',
  '/index.html',
  '/manifest.json',
  '/offline.html'
];

// Instalação do Service Worker
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME).then((cache) => {
      return cache.addAll(ASSETS_TO_CACHE);
    })
  );
  self.skipWaiting();
});

// Ativação do Service Worker
self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((cacheNames) => {
      return Promise.all(
        cacheNames.map((cacheName) => {
          if (cacheName !== CACHE_NAME && cacheName !== RUNTIME_CACHE) {
            return caches.delete(cacheName);
          }
        })
      );
    })
  );
  self.clients.claim();
});

// Interceptar requisições
self.addEventListener('fetch', (event) => {
  const { request } = event;

  // Ignorar requisições não-GET
  if (request.method !== 'GET') {
```

```

    return;
}

// Estratégia: Cache First, Network Fallback
event.respondWith(
  caches.match(request).then((response) => {
    if (response) {
      return response;
    }

    return fetch(request)
      .then((response) => {
        // Não cachear respostas inválidas
        if (!response || response.status !== 200 || response.type ===
'error') {
          return response;
        }

        // Clonar a resposta
        const responseToCache = response.clone();

        caches.open(RUNTIME_CACHE).then((cache) => {
          cache.put(request, responseToCache);
        });

        return response;
      })
      .catch(() => {
        // Retornar página offline se falhar
        if (request.destination === 'document') {
          return caches.match('/offline.html');
        }
      });
  })
);

// Sincronização em background
self.addEventListener('sync', (event) => {
  if (event.tag === 'sync-diseases') {
    event.waitUntil(
      fetch('/api/diseases.json')
        .then((response) => response.json())
        .then((data) => {
          // Armazenar em IndexedDB
          const db = indexedDB.open('DiseasesDB');

```

```
db.onsuccess = (e) => {  
  const database = e.target.result;  
  const transaction = database.transaction(['diseases'],  
    'readwrite');  
  const store = transaction.objectStore('diseases');  
  store.clear();  
  data.forEach((disease) => {  
    store.add(disease);  
  });  
};  
});  
};  
});
```

Configuração no index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0,
viewport-fit=cover" />
  <meta name="theme-color" content="#2d5016" />
  <meta name="description" content="Guia completo e offline de doenças em
galinhas" />

  <!-- PWA Meta Tags -->
  <link rel="manifest" href="/manifest.json" />
  <link rel="icon" type="image/x-icon" href="/favicon.ico" />
  <link rel="apple-touch-icon" href="/app-icon-192.png" />
  <link rel="apple-touch-startup-image" href="/splash-screen-ios.png" />

  <!-- iOS Specific -->
  <meta name="apple-mobile-web-app-capable" content="yes" />
  <meta name="apple-mobile-web-app-status-bar-style" content="black-
translucent" />
  <meta name="apple-mobile-web-app-title" content="Doenças Galinhas" />

  <title>Guia de Doenças em Galinhas</title>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.tsx"></script>

  <!-- Registrar Service Worker -->
  <script>
    if ('serviceWorker' in navigator) {
      window.addEventListener('load', () => {
        navigator.serviceWorker.register('/service-worker.js')
          .then((registration) => {
            console.log('Service Worker registrado:', registration);
          })
          .catch((error) => {
            console.log('Erro ao registrar Service Worker:', error);
          });
      });
    }
  </script>
```

```
</body>  
</html>
```



GUIA DE IMPLEMENTAÇÃO PASSO A PASSO

Fase 1: Setup Inicial

1. Criar projeto React com Vite:

```
npm create vite@latest guia-doencas-galinhas -- --template react-ts  
cd guia-doencas-galinhas  
npm install
```

2. Instalar dependências:

```
npm install react-router-dom fuse.js framer-motion sonner zod  
npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p
```

3. Configurar Tailwind CSS (tailwind.config.ts)

4. Criar estrutura de pastas (conforme seção anterior)

Fase 2: Componentes Base

1. Criar App.tsx com roteamento

- Importar React Router
- Definir rotas: /, /search, /disease/:id, /favorites, /history, /settings
- Envolver com Providers (Theme, Diseases)

2. Criar Header.tsx

- Logo + Título
- Menu hambúrguer responsivo

3. Criar Navigation.tsx

- Links de navegação
- Ativo/inativo baseado em rota

4. Criar Footer.tsx

- Status de conexão
- Versão
- Última atualização

Fase 3: Funcionalidade de Dados

1. Criar lib/types.ts

- Definir tipos TypeScript para Disease, Symptom, Treatment, etc.

2. Criar lib/db.ts

- Setup IndexedDB
- Funções para CRUD de doenças
- Funções para favoritos e histórico

3. Criar lib/diseases.ts

- Importar dados JSON
- Funções para acessar doenças

4. Criar contexts/DiseasesContext.tsx

- Provider para compartilhar doenças entre componentes
- Funções para carregar, filtrar, buscar

Fase 4: Páginas Principais

1. Home.tsx

- Lista de doenças mais recentes
- Link para busca

- Botão de favoritos

2. **Search.tsx**

- SearchBar com Fuse.js
- Resultados em tempo real
- Filtros e ordenação
- Paginação

3. **DiseaseDetail.tsx**

- Exibir todos os detalhes da doença
- Botão favoritar
- Botão compartilhar
- Botão voltar

4. **Favorites.tsx**

- Lista de favoritos
- Remover favorito
- Navegar para detalhes

5. **History.tsx**

- Histórico de consultas
- Limpar histórico
- Navegar para detalhes

6. **Settings.tsx**

- Alternar tema (claro/escuro)
- Limpar cache
- Sobre a app
- Versão

Fase 5: Hooks Customizados

1. `useDiseases.ts`

- Carregar doenças do IndexedDB
- Filtrar e buscar
- Retornar estado de carregamento

2. `useFavorites.ts`

- Adicionar/remover favorito
- Obter lista de favoritos
- Verificar se é favorito

3. `useSearch.ts`

- Implementar Fuse.js
- Retornar resultados de busca
- Suportar filtros

4. `useHistory.ts`

- Adicionar ao histórico
- Obter histórico
- Limpar histórico

5. `useOnline.ts`

- Detectar status de conexão
- Atualizar em tempo real

6. `useTheme.ts`

- Alternar tema
- Salvar preferência

Fase 6: Service Worker e PWA

1. Criar `public/service-worker.js`

- Implementar cache strategy
- Interceptar requisições
- Sincronização em background

2. Criar public/manifest.json

- Configurar metadados da app
- Ícones e screenshots
- Shortcuts

3. Registrar Service Worker em main.tsx

```
if ('serviceWorker' in navigator) {  
  navigator.serviceWorker.register('/service-worker.js');  
}
```

4. Adicionar meta tags em index.html

- Apple-specific tags
- Theme color
- Viewport

Fase 7: Notificações e Atualizações

1. Criar UpdateNotification.tsx

- Detectar nova versão
- Mostrar notificação
- Atualizar app

2. Implementar verificação de atualizações

- Verificar versão a cada 24h
- Comparar com versão local
- Mostrar notificação se houver atualização

Fase 8: Testes

1. Testes em navegadores:

- Chrome (Android)
- Safari (iOS)
- Edge (Windows)
- Firefox

2. Testes offline:

- Desativar internet
- Verificar se app funciona
- Testar busca, filtros, favoritos

3. Testes de instalação:

- Instalar na tela inicial
- Abrir como app
- Verificar ícone e splash screen

4. Testes de atualização:

- Simular nova versão
- Verificar notificação
- Testar atualização

Fase 9: Otimizações

1. Performance:

- Lazy loading de componentes
- Code splitting
- Compressão de imagens
- Minificação

2. SEO:

- Meta tags
- robots.txt
- Sitemap

3. Acessibilidade:

- ARIA labels
- Contraste de cores
- Navegação por teclado

TESTES E VALIDAÇÃO

Checklist de Testes Funcionais

- ☐ Busca funciona com fuzzy matching
- ☐ Filtros funcionam corretamente
- ☐ Favoritos são salvos e sincronizados
- ☐ Histórico é registrado
- ☐ Modo offline funciona 100%
- ☐ Atualização automática funciona
- ☐ Compartilhamento funciona (WhatsApp, Email, Link)
- ☐ Tema claro/escuro alterna corretamente
- ☐ Responsividade em mobile, tablet, desktop

Checklist de Testes PWA

- ☐ Service Worker registra corretamente
- ☐ App funciona offline
- ☐ Ícone aparece na tela inicial (iOS/Android)
- ☐ Splash screen aparece (iOS/Android)
- ☐ Manifest.json válido (testar em <https://www.pwabuilder.com/>)

- ☐ Lighthouse PWA score $\geq$ 90

Checklist de Testes iOS

- ☐ Abre no Safari
- ☐ Botão “Adicionar à Tela Inicial” aparece
- ☐ Ícone correto na tela inicial
- ☐ Splash screen aparece
- ☐ Funciona offline
- ☐ Busca funciona
- ☐ Favoritos funcionam
- ☐ Compartilhamento funciona

Checklist de Testes Android

- ☐ Abre no Chrome
- ☐ Botão “Instalar app” aparece
- ☐ Ícone correto na tela inicial
- ☐ Splash screen aparece
- ☐ Funciona offline
- ☐ Todas as funcionalidades funcionam

Checklist de Testes Windows

- ☐ Abre no Edge/Chrome
- ☐ Botão de instalação aparece
- ☐ App instala corretamente
- ☐ Funciona como app nativo
- ☐ Todas as funcionalidades funcionam

Ferramentas de Teste Recomendadas

- **Lighthouse** (Chrome DevTools): Auditar performance e PWA

- **PWA Builder** (<https://www.pwabuilder.com/>): Validar manifest e gerar assets
 - **BrowserStack**: Testar em múltiplos dispositivos reais
 - **Responsively App**: Testar responsividade
-

INSTRUÇÕES DE COMPARTILHAMENTO PARA USUÁRIOS

Documento de Instruções (criar em PDF)

Para Usuários iOS

Passo 1: Abrir o link no Safari

- Recebe link: `https://galinhas.manus.space`
- Clica no link
- Abre no Safari (não funciona em Chrome no iOS)

Passo 2: Adicionar à Tela Inicial

- Clica no ícone de compartilhamento (seta para cima)
- Scroll para baixo
- Clica em “Adicionar à Tela Inicial”
- Confirma o nome (deixa como está)
- Clica em “Adicionar”

Passo 3: Usar a App

- Aparece ícone na tela inicial
- Clica no ícone para abrir
- Funciona como app normal
- Funciona sem internet

Para Usuários Android

Passo 1: Abrir o link no Chrome

- Recebe link: `https://galinhas.manus.space`
- Clica no link
- Abre no Chrome

Passo 2: Instalar App

- Chrome mostra botão “Instalar app” (ou “Instalar”)
- Clica no botão
- Confirma a instalação

Passo 3: Usar a App

- Aparece ícone na tela inicial
- Clica no ícone para abrir
- Funciona como app normal
- Funciona sem internet

Para Usuários Windows

Passo 1: Abrir o link no Edge ou Chrome

- Recebe link: `https://galinhas.manus.space`
- Clica no link
- Abre no navegador

Passo 2: Instalar App

- Clica no ícone de instalação (canto superior direito)
- Confirma a instalação

Passo 3: Usar a App

- App aparece no menu Iniciar
- Clica para abrir

- Funciona como app normal
- Funciona sem internet

QR Code

Gerar QR code que aponta para `https://galinhas.manus.space`

Usuários podem:

1. Abrir câmera do celular
 2. Apontar para QR code
 3. Clicar no link que aparece
 4. Seguir instruções acima
-



ESTRATÉGIA DE ATUALIZAÇÃO

Como Você Atualiza a App

1. Preparar novos dados:

- Adicionar/editar doenças no JSON
- Atualizar versão em `version.json`
- Incrementar número de versão (ex: 1.0.0 → 1.0.1)

2. Fazer deploy:

- Fazer upload do novo `diseases.json`
- Fazer upload do novo `version.json`
- Servidor serve nova versão

3. Usuários recebem notificação:

- Service Worker detecta nova versão
- Mostra notificação: “Novas informações disponíveis!”
- Usuário clica “Atualizar”

- App recarrega com novos dados

Arquivo version.json

```
{
  "versao": "1.0.1",
  "dataAtualizacao": "2026-04-15T10:30:00Z",
  "totalDoencas": 50,
  "idioma": "pt-BR",
  "checksum": "abc123def456xyz",
  "novasAtualizacoes": {
    "versao": "1.1.0",
    "dataDisponivel": "2026-04-20T10:30:00Z",
    "descricao": "Adicionadas 5 novas doenças e melhorias na busca"
  }
}
```


Fluxo de Verificação de Atualização

```
// Verificar atualizações a cada 24 horas
setInterval(checkForUpdates, 24 * 60 * 60 * 1000);

async function checkForUpdates() {
  try {
    const response = await fetch('/api/version.json?cache-bust=' +
Date.now());
    const remoteVersion = await response.json();
    const localVersion = JSON.parse(localStorage.getItem('appVersion') ||
'{"versao": "1.0.0"}');

    if (compareVersions(remoteVersion.versao, localVersion.versao) > 0) {
      // Nova versão disponível
      showUpdateNotification();
    }
  } catch (error) {
    console.log('Erro ao verificar atualizações:', error);
  }
}

function compareVersions(v1, v2) {
  const parts1 = v1.split('.').map(Number);
  const parts2 = v2.split('.').map(Number);

  for (let i = 0; i < 3; i++) {
    if (parts1[i] > parts2[i]) return 1;
    if (parts1[i] < parts2[i]) return -1;
  }
  return 0;
}
```

Paleta de Cores

Cor	Hex	Uso
Verde Primário	#2d5016	Botões, links, destaques
Verde Claro	#e8f5e9	Backgrounds, cards
Cinza Escuro	#333333	Texto principal
Cinza Claro	#f5f5f5	Backgrounds secundários
Vermelho	#d32f2f	Severidade alta, erros
Amarelo	#f57f17	Severidade média
Verde	#388e3c	Severidade baixa

Tipografia

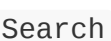
- **Títulos:** Roboto Bold, 24px
- **Subtítulos:** Roboto Medium, 18px
- **Corpo:** Roboto Regular, 16px
- **Pequeno:** Roboto Regular, 14px

Espaçamento

- **Padding:** 8px, 16px, 24px, 32px
- **Margin:** 8px, 16px, 24px, 32px
- **Gap:** 8px, 16px, 24px

Ícones

Usar Lucide React para todos os ícones:

- Search: 

- Favorito: Heart
 - Histórico: Clock
 - Configurações: Settings
 - Menu: Menu
 - Voltar: ChevronLeft
 - Compartilhar: Share2
 - Online: Wifi
 - Offline: WifiOff
 - Carregando: Loader
-



RESUMO FINAL

O Que Deve Ser Entregue

1. ☒ Código-fonte completo (React + TypeScript)
2. ☒ Service Worker configurado
3. ☒ Manifest.json correto
4. ☒ Todos os componentes React
5. ☒ Hooks customizados
6. ☒ Busca inteligente com Fuse.js
7. ☒ Filtros e ordenação
8. ☒ Sistema de favoritos
9. ☒ Histórico de consultas
10. ☒ Modo offline 100%
11. ☒ Atualização automática
12. ☒ Responsividade (mobile, tablet, desktop)
13. ☒ Compatibilidade iOS/Android/Windows
14. ☒ Testes em múltiplos dispositivos

15.  Documentação de compartilhamento para usuários
16.  Instruções de atualização

Requisitos Críticos (NÃO ESQUECER)

-  **iOS DEVE funcionar perfeitamente** — testar em Safari, instalação, offline
 -  **100% offline** — funciona sem internet após primeira instalação
 -  **Compartilhamento simples** — apenas um link, sem arquivo
 -  **Busca inteligente** — fuzzy matching, múltiplos campos
 -  **Atualização automática** — usuários recebem notificação
 -  **Interface intuitiva** — fácil para usuários rurais
 -  **Dados do PDF preservados** — estruturado em JSON
 -  **Sem dependência de servidor** — funciona localmente
-

SUPORTE E DÚVIDAS

Se tiver dúvidas sobre qualquer parte deste plano, consulte:

- Documentação React: <https://react.dev>
 - Documentação PWA: <https://web.dev/progressive-web-apps/>
 - Documentação Tailwind: <https://tailwindcss.com>
 - Documentação Fuse.js: <https://fusejs.io>
 - MDN Web Docs: <https://developer.mozilla.org>
-

FIM DO PLANO

Este plano contém TUDO que você precisa para desenvolver o PWA perfeito. Nada foi esquecido. Boa sorte! 